

A world map where the continents are formed by a dense collection of coins of various denominations and colors (gold, silver, copper). The background is a gradient of green and blue, overlaid with faint, semi-transparent circular patterns and lines, suggesting a global or financial theme.

CROWDFUNDING

BY
TEAM NEXUS

AGENDA

1. Team Member Roles | Improvements | Responsibilities
 2. Problem Statement | Project Description | Team Working Agreement
 3. User Personas | MVP | Project Design
 4. Technologies | Algorithms | Diagrams
 5. Product Backlog
 6. Metrics
 7. Sprint 1
 - User Stories
 - Test Cases
 - Stories Completed
 - Stories Not Completed
-
1. Metrics
 - Charts
-
1. Retrospective
 2. Sprint 2
 3. Project Demo

GETTING TO KNOW THE TEAM...



Ashwanth Maram
Full Stack Developer



Ashwitha Maram
Developer/Tester



Raj Mandaliya
Block Chain Developer

GETTING TO KNOW THE TEAM...



Vinukonda Yashwanth Reddy
Backend and Database Engineer



Nyi Nyi Lwin
Full Stack Developer



Naveen Pesalabanda
SCRUM Master

IMPROVEMENTS

- Have Team Member introduction before starting the presentation and Retrospective.
- Add group email to Wiki page link
- Add Project description chart to Presentation slides.

PROJECT DESCRIPTION

Project Name:	Crowd Funding
Team:	Team Nexus
Project Description:	It's a platform that helps people raise money safely and fairly using blockchain, making sure all funds are transparent and controlled by the community. For campaign creators and contributors who seek a transparent, secure, and fair way to raise and fund projects, the Crowd Funding platform is a blockchain-based fundraising solution that enables campaign creation, contributor governance, and automatic refunds. Unlike traditional centralized platforms like Kickstarter or GoFundMe, our application uses smart contracts to eliminate intermediaries, reduce fees, and ensure community-driven decision-making.
Benefit Outcomes:	Transparent funds – everyone can see usage Secure money handling – via blockchain Community voting – contributors decide Automatic refunds – if project fails Low fees – cheaper than traditional platforms
GitHub Link:	https://github.com/htmwf2025-Team2/wiki

TEAM WORKING AGREEMENT

Communication Expectations

- Use Zoom, GitHub for collaboration.
- Respect everyone opinions; decisions are made collectively.
- Notifying the team in advance if you cannot complete or attend.
- In case of any conflicts, present research-backed solutions and resolve by majority vote.
- Keeping everyone updated when pushing code or making changes.
- No last-minute dumping of work on others.

Meeting Schedules

- Meetings will be held twice weekly on Zoom or Google meet
- Meeting time is set based on availability.
- Agenda will be shared in advance for preparation.
- Sprint Planning at the start, Retrospective at the end of each sprint.
- Absence must be informed ahead of time; member must read meeting summary.
- All members are expected to participate and contribute fairly.

TEAM WORKING AGREEMENT

Team Dynamics & Principles

- Each member must clearly understand and own their tasks.
- Workload distributed fairly; large tasks broken into smaller ones.
- Every task has a **deadline** and review on completion.
- Encourage problem-solving before escalating blockers.
- Trackers must be updated regularly.
- Work progress should be committed and shared on GitHub.

Checklist for Task Completion

- Code should run **without errors**.
- Work must meet all **acceptance criteria**.
- Each task is **reviewed and tested** by everyone
- Documentation should be updated with the task.
- Final review by all team members before submission.

THE STARTUP FOUNDER PERSONA

Name: Alice Johnson

Age: 28

Gender: Female

Occupation: Startup Founder

Alice is an ambitious entrepreneur developing a sustainable fashion brand. She wants to raise money to bring her eco-friendly products to market but struggles with high fees and limited visibility on traditional crowdfunding platforms.

Challenges:

- Struggles to gain trust from contributors due to lack of transparency.
- Centralized platforms charge high fees, cutting into her funding.
- Limited control over how her campaign is presented.

Goals:

- Raise funds fairly and transparently.
- Build credibility with backers.
- Ensure contributors see exactly how their money is being used.



THE INVESTOR PERSONA

Name: Bob Smith

Age: 35

Gender: Male

Occupation: Investor

Bob is a passionate investor who likes to fund innovative projects. He wants to support creators but often worries about whether his money is being used properly and feels disconnected after contributing.

Challenges:

- Unsure if funds are spent transparently
- No refunds if a project fails
- Has no control once money is invested

Goals:

- Invest with full transparency and security
- Receive refunds if funding goals aren't met
- Have a say in how project funds are used through voting



THE COMMUNITY MEMBER PERSONA

Name: Cathy Lee
Age: 22
Gender: Female
Occupation: College Student / Community Member

Cathy is an active member of online communities and DAOs. She enjoys contributing to causes but wants her voice to count, not just her money.

Challenges:

- No say in how project funds are allocated
- Hesitant to contribute without accountability
- Finds most platforms profit-driven and restrictive

Goals:

- Participate in voting and decision-making
- Support transparent, community-driven projects
- Learn about blockchain while backing meaningful causes



MVP

- **Campaign Creation:** Allow users to launch funding campaigns with metadata (title, goal, deadline).
- **Contribution Logic:** Accept ETH contributions and track them per user.
- **Refund Mechanism:** Automatically refund contributors if the goal isn't met.
- **Governance Voting:** Let contributors vote on fund allocation or milestones.

TOOLS & TECHNOLOGIES

- **Blockchain:** Ethereum + Solidity
- **Development:** Remix IDE, Hardhat
- **Network:** MetaMask, Sepolia Test Network, Sepolia ETH Faucets
- **Deployment & Testing:** Hardhat
- **Collaboration Tools:** GitHub, VS Code



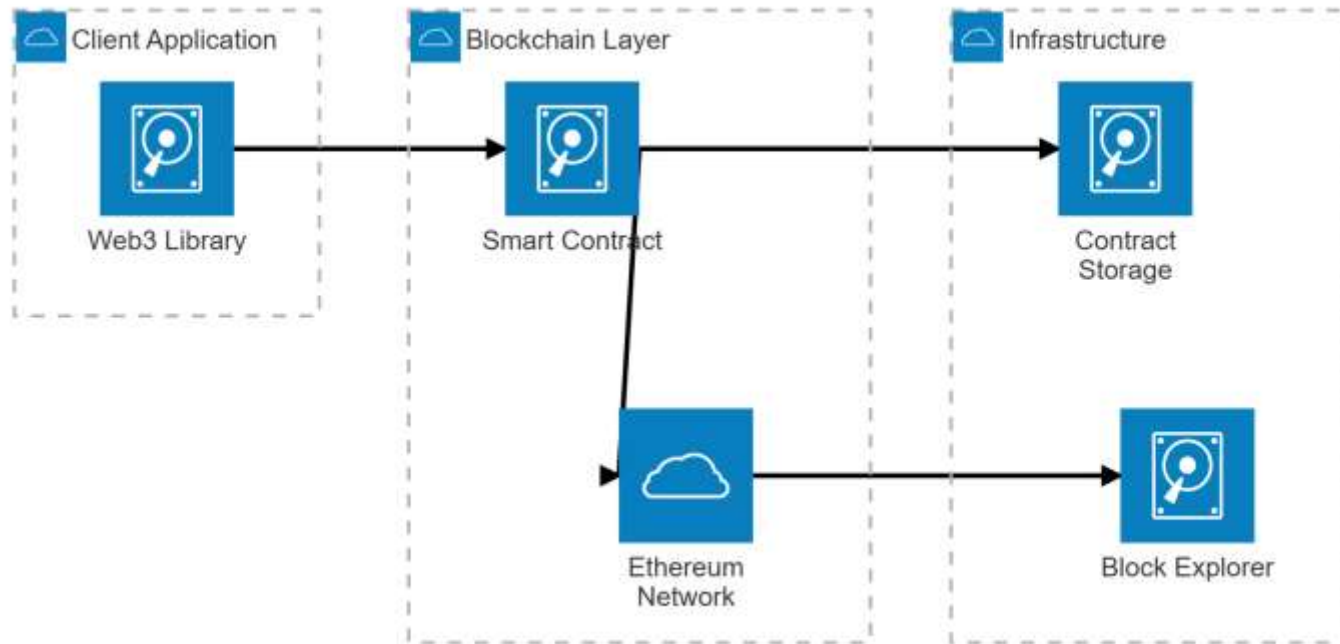
ALGORITHMS & LOGIC

- **Smart Contract–Based Voting Mechanism**
Implements a weighted voting system based on user stake or role.
Ensures tamper-proof, transparent decision-making using smart contracts.
- **Refund Logic**
Automatically refunds users if the funding goal is not achieved.
Executed via smart contract conditions to ensure user trust.
- **Fraud Detection Rules** (*Optional*)
Detects suspicious behavior like duplicate voting or fake contributions.
Helps maintain fairness and integrity in decentralized processes.

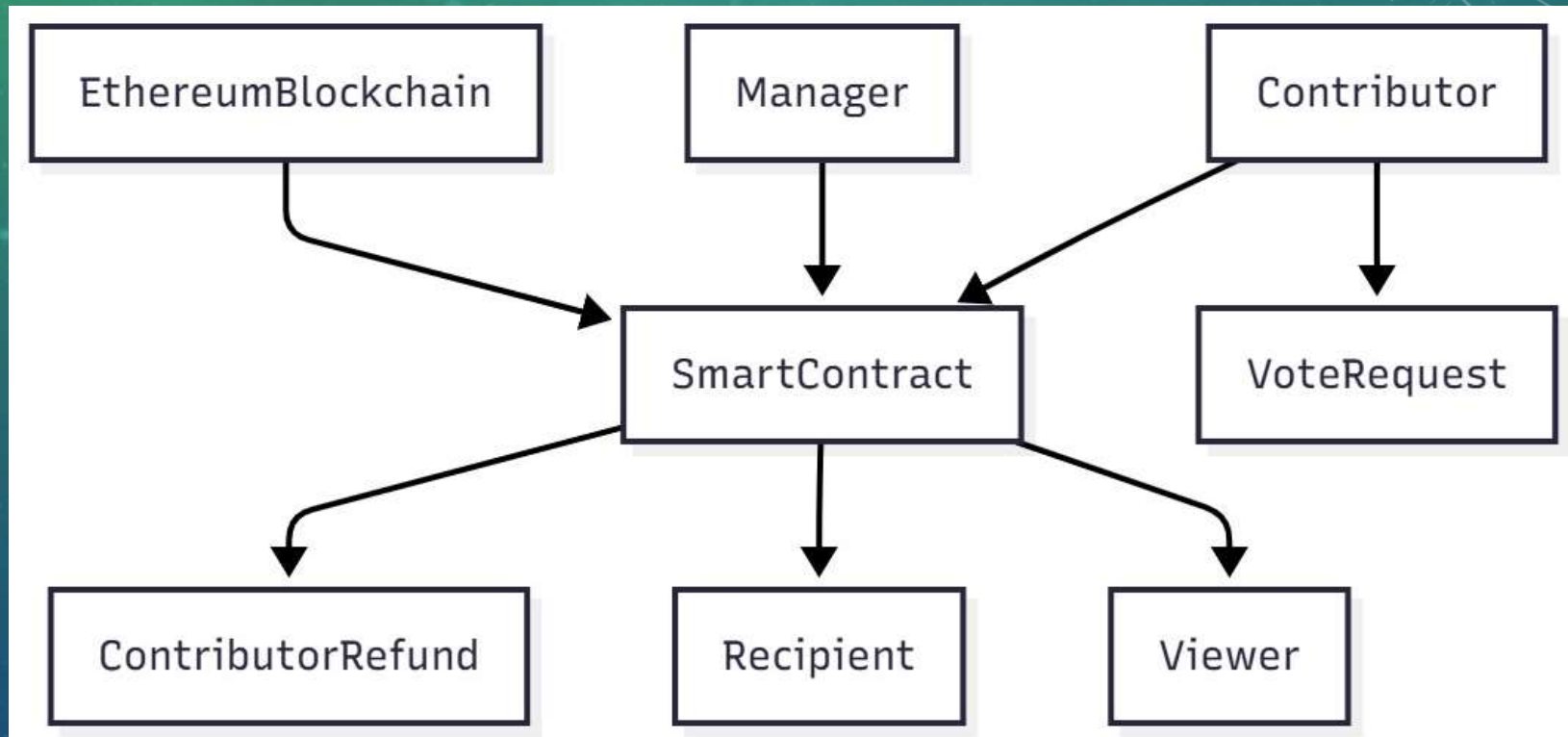
Diagrams

The background is a gradient of green and blue, transitioning from a lighter green at the top to a darker blue at the bottom. It is covered with numerous small white dots, creating a starry or particle-like effect. In the corners, there are faint, stylized circular diagrams. The top-left corner shows a partial circle with an arrow. The top-right corner features a large, detailed circular diagram with concentric rings, radial lines, and numerical markings (0, 90, 180, 270). The bottom-right corner has a circular diagram with concentric rings and an arrow. The bottom-left corner shows a partial circular diagram with an arrow.

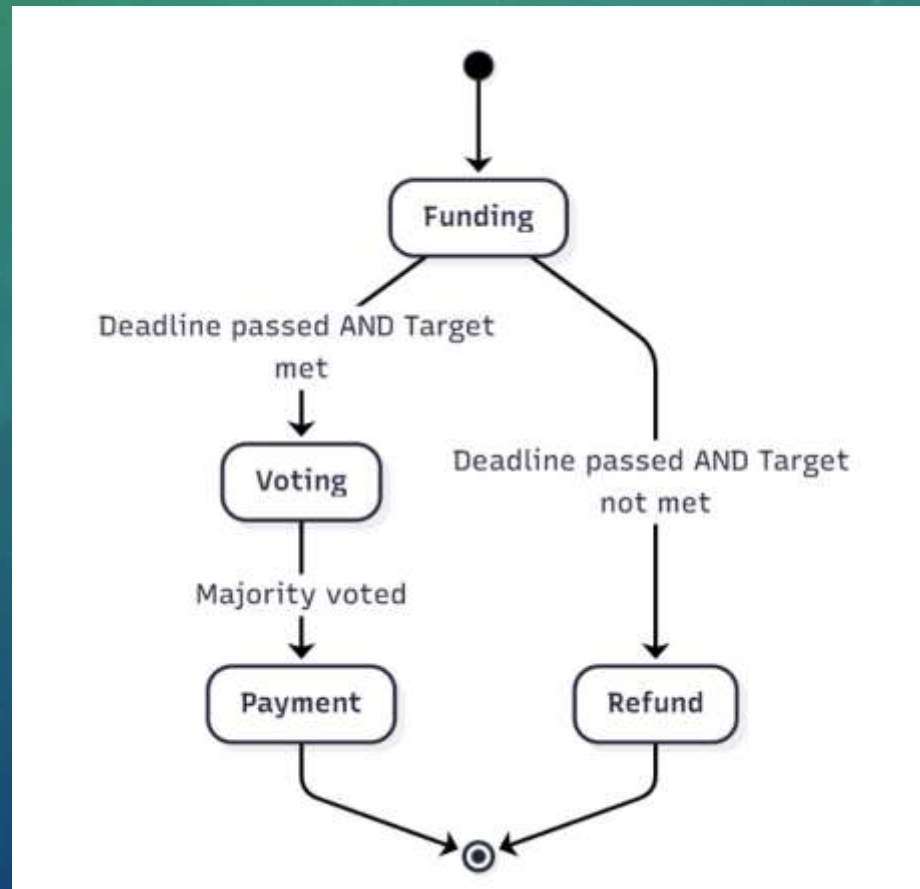
ARCHITECTURE DIAGRAM



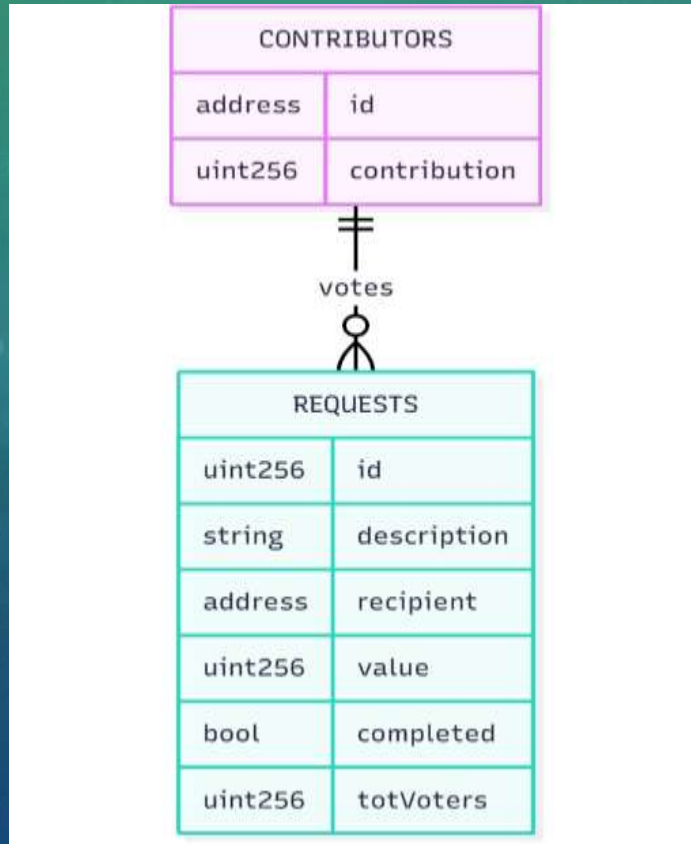
CONTEXT DIAGRAM



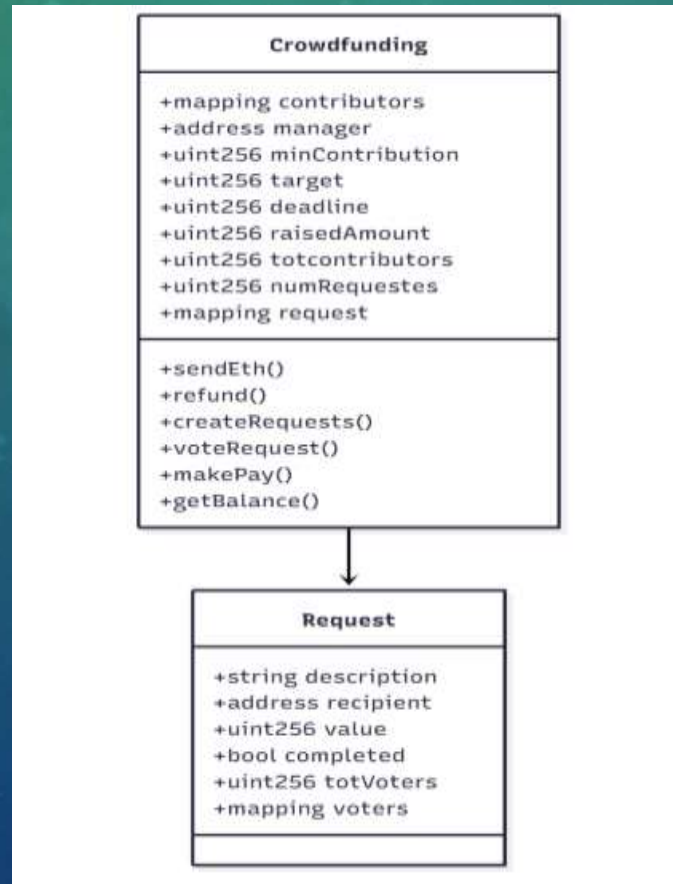
STATE DIAGRAM



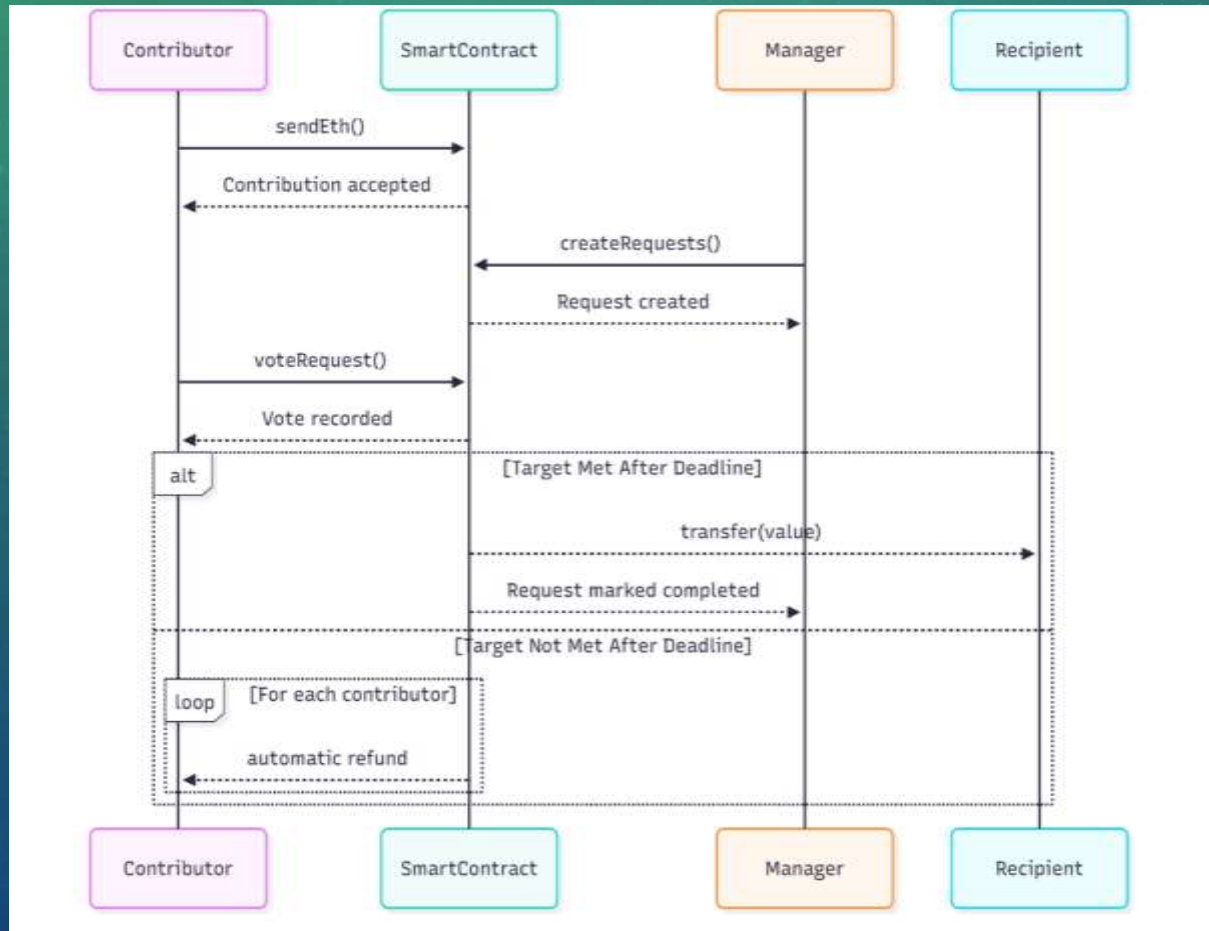
ER DIAGRAM



CLASS DIAGRAM



SEQUENCE DIAGRAM



PRODUCT BACKLOG



No.	User Stories	Acceptance Criteria	Feature	Component	Story Points
US_01	As a campaign manager, I want to create new campaigns with a target amount and deadline	- The system allows manager to set target and deadline - Contract deploys successfully - Manager automatically becomes campaign owner	Campaign Creation	Smart Contract	25
US_02	As a supporter, I want to contribute funds to a campaign	- Contribution $\geq$ min amount - Contributor stored - Total raised updates automatically	Campaign Contribution	Smart Contract	25
US_03	As a contributor, I want to get a refund if the goal is not met before the deadline	- Refund available only after deadline - Only contributors can request refund - ETH refunded equals original contribution	Refund Processing	Smart Contract	25
US_04	As a manager, I want to create spending requests for raised funds	- Manager creates request with description, value, recipient - Default state: not completed	Request Creation	Governance	20
US_05	As a contributor, I want to vote on spending requests	- Only contributors can vote - Each contributor can vote once - Vote count updates accurately	Request Voting	Governance	22

US_06	As a manager, I want to release funds after majority approval	- Only manager executes payments - Majority approval required - ETH transferred - Request marked completed	Fund Release	Governance	25
US_07	As a user, I want to view all campaigns with details	- Display campaign list - Show goal, deadline, raised amount - Reflect status dynamically	Campaign Listing	Data Display	20
US_08	As a user, I want to see campaign status (Active, Successful, Failed)	- Status updates based on raised amount and time - Proper label displayed	Campaign Status	Data Display	20
US_09	As a system, I want to validate minimum contribution to ensure fairness	- Reject transactions < min amount - Display error message	Validation Logic	Contract Validation	10

PRODUCT BACKLOG OVERVIEW

Sprint 1 (60 pts)	Sprint 2 (67 pts)	Sprint 3 (65 pts)
US_01: As a Campaign Manager, I want to create new campaigns with a goal and deadline.	US_04: As a Manager, I want to create spending requests for raised funds.	US_03: As a Contributor, I want to get a refund if the goal is not met.
US_02: As a Supporter, I want to contribute funds to campaigns.	US_05: As a Contributor, I want to vote on spending requests.	US_07: As a User, I want to view all campaigns with details.
US_09: As a System, I want to validate minimum contribution to ensure fairness.	US_06: As a Manager, I want to release funds after majority approval.	US_08: As a User, I want to see campaign status (Active, Successful, Failed).

SPRINT 1

SPRINT 1 BACKLOG

No.	User Stories	Acceptance Criteria	Feature	Component	Story Points	Assigned To
US_01	As a Campaign Manager, I want to create new campaigns with a target amount and deadline.	• System allows manager to set target and deadline • Contract deploys successfully • Manager automatically becomes campaign owner	Campaign Creation	Smart Contract	25	Ashwitha Ashwanth
US_02	As a Supporter, I want to contribute funds to a campaign.	• Contribution $\geq$ minimum amount • Contributor stored • Total raised updates automatically	Campaign Contribution	Smart Contract	25	Nyi Nyi Raj
US_09	As a System, I want to validate minimum contribution to ensure fairness.	• Reject transactions $<$ minimum amount • Display error message	Validation Logic	Contract Validation	10	Naveen Yashwanth

SPRINT 1 BACKLOG TESTCASES

Feature to Test	Test Case	Test Steps	Test Data	Expected Result	Actual Result	Pass/Fail
Campaign Creation	Verify campaign deployment	Connect MetaMask > Deploy contract with target and deadline	Target = 1000 ETH, Deadline = 5 min	Contract deployed successfully and manager = sender address	Contract deployed successfully and manager address displayed	Pass
Campaign Creation	Validate manager ownership	After deployment, check manager()	-	Returns deployer address only	Deployer address returned correctly	Pass
Campaign Creation	Verify target and deadline recorded	Check target() and deadline() values	Target = 1000 wei, Deadline +60s	Values recorded correctly	Displayed correctly in Remix	Pass
Contribution	Accept valid contribution (≥ 100 wei)	Call sendEth() with value = 100 wei	100 wei	Raised amount $\neq 100$; totcontributors $\neq 1$	Transaction confirmed and balance updated	Pass
Contribution	Reject low contribution (< 100 wei)	Call sendEth() with value = 10 wei	10 wei	Transaction reverts with 'Minimum contribution not met'	Error displayed correctly	Pass
Contribution	Reject after deadline	Fast-forward time > deadline > call sendEth()	100 wei	Transaction reverts with 'You have missed the time'	Reverted as expected	Pass
Contribution	Multiple contributors update total	Send funds from two different accounts	100 wei each	Total contributors = 2; Raised = 200 wei	Values matched expected output	Pass
Validation Logic	Ensure balance matches raisedAmount	Check getBalance() vs raisedAmount()	After multiple contributions	Both values equal	Matched exactly	Pass
Validation Logic	Ensure only contributors can vote	Call voteRequest() without contribution	-	Reverts with 'You are not contributor'	Reverted as expected	Pass

STORIES COMPLETED

No.	User Stories	Completed Date	Story Points	Completed Points
US_01	As a campaign manager, I want to create new campaigns with a target amount and deadline.	Oct 14	25	25
	Implement contract deployment with target and deadline.	Oct 14	10	10
	Verify manager ownership after deployment.	Oct 15	5	5
	Validate stored target and deadline values.	Oct 15	10	10
US_02	As a supporter, I want to contribute funds to a campaign.	Oct 17	25	25
	Test minimum contribution (≥ 100 wei).	Oct 17	10	10
	Reject invalid contribution (< 100 wei).	Oct 18	5	5
	Verify raised amount and contributors count update.	Oct 18	5	5
US_03	As a contributor, I want to get a refund if the goal is not met before the deadline.	Oct 20	10	10
	Ensure refund is available only after deadline and target unmet.	Oct 20	5	5
	Confirm only contributors can initiate refunds.	Oct 21	5	5

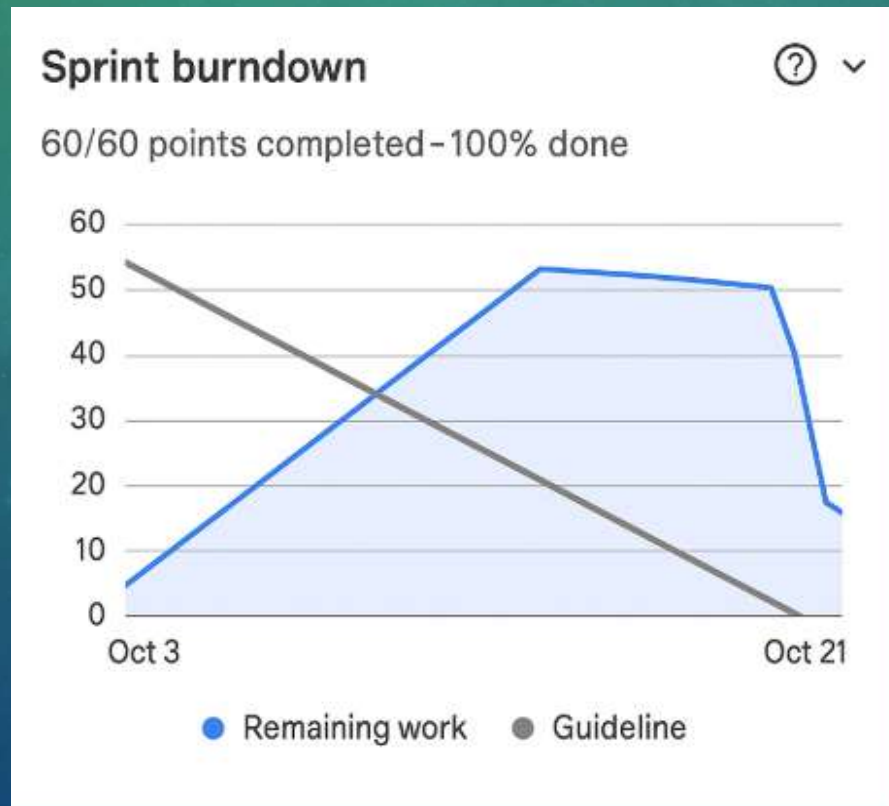
METRICS: SPRINT 1

Story ID	Story description	Story points estimated	Story points delivered
US_01	As a campaign manager, I want to create new campaigns with a target amount and deadline	25	25
US_02	As a supporter, I want to contribute funds to a campaign	25	25
US_09	As a contributor, I want to get a refund if the goal is not met before the deadline	10	10

METRICS: TEAM VELOCITY

Metric	Value
Total estimated points:	60
Total points delivered:	60
Points carried over:	0
Team Velocity:	60

METRICS: BURNDOWN CHART



SPRINT 1 RETROSPECTIVE

What went well?

- Improved Meeting Structure
- Efficient Time Management
- Better Communication
- Seamless Code Integration
- Open and Honest Communication

What could be improved?

- Confidence in Decision-Making
- Faster Response Times
- Better Way of Scheduling
- Clearer Dependency Tracking
- Thorough Technical Analysis
- Smaller, Manageable Tasks

Our commitments

- Engage More in Technical Discussions
- Improve Task & Dependency Tracking
- Enhance Team Coordination
- Frequent Jira Updates

SPRINT 2 PLANNING

No.	User Stories	Acceptance Criteria	Feature	Component	Story Points	Assigned To
US_04	As a Manager, I want to create spending requests for raised funds.	• Manager creates request with description, value, recipient address • Default state: not completed	Request Creation	Governance	25	Ashwanth Yashwanth
US_05	As a Contributor, I want to vote on spending requests.	• Only contributors can vote • Each contributor can vote once • Vote count updates accurately	Request Voting	Governance	22	Nyi Raj
US_06	As a Manager, I want to release funds after majority approval.	• Only manager executes payments • Majority approval required • ETH transferred • Request marked completed	Fund Release	Governance	20	Ashwitha Naveen

SPRINT-2 STORIES PLANNED

No.	User Story	Acceptance Criteria / Planned Tasks	Planned Date	Story Points
US_04	As a Manager, I want to create spending requests for raised funds.	• Manager can create request with description, recipient and value • Request stored on contract and visible to contributors • Initial state = Pending	Oct 22 – Oct 25	25
US_05	As a Contributor, I want to vote on spending requests.	• Only contributors can vote • One vote per contributor • Majority approval required	Oct 24 – Oct 26	22
US_06	As a Manager, I want to release funds after majority approval.	• Manager executes payment after approval • Funds transferred to recipient • Request marked as completed	Oct 26 – Oct 29	20

PROJECT DEMO

The screenshot displays the REMIX IDE v1.1.0 interface. The top navigation bar includes links to Gmail, YouTube, Maps, Home | StackUp, DeFiLlama - DeFi Da..., GitHub - OpenZepp..., FURUCOMBO /Crea..., Maximizing Your Ar..., The Complete Block..., slither/README.md..., and All Bookmarks. The main workspace is titled 'default_workpace' and contains a file named 'Crowdfunding.sol'. The code in the editor is as follows:

```
21 mapping(uint256 => Request) public request;
22 uint256 public numRequests;
23
24 constructor(uint256 _target, uint256 _seconds) {
25     target = _target;
26     deadline = block.timestamp + _seconds;
27     minContribution = 100 wei;
28     manager = msg.sender;
29 }
```

On the left sidebar, the 'DEPLOY & RUN TRANSACTIONS' panel is active. It shows 'Deployed Contracts' with a list containing 'CROWDFUNDING AT 0x08B...'. Below this, the 'CREATEREQUESTS' section is expanded, showing a form with fields for 'description' (Daycare), 'recipients' (0xd8b04580fc35a1185809073a4c...), and 'value' (100). The 'makePay' button is highlighted. Below the form, there are buttons for 'refund', 'sendEth', 'withdraw', 'contribute', 'deadline', and 'getBalance'.

The bottom panel shows the 'Explain contract' section with a search bar and a 'Filter with transaction hash or address' input. Below this, a transaction is listed with the following details:

- Transaction hash: 0x740145c4395804f0e4d0804f1a2aef21740278014c7a27194d09058
- Block hash: 0x10c090278485d7003aef2b0c7545504acc0f12043a1077884e17d210a04b
- Block number: 4
- From: 0x50385a6701c560740cf807fcb079f906e4c4
- To: Crowdfunding.createRequests(string,address,uint256) 0xd8b04580fc35a1185809073a4c40a2811fa0
- Gas: 117463 gas
- Transaction cost: 11055 gas

The transaction status is 'Success' and the execution was successful.

PROJECT DEMO

The screenshot displays the Remix IDE interface, a web-based development environment for Ethereum. The top bar shows the Remix logo and version (v1.1.0). The left sidebar contains a 'DEPLOY & RUN TRANSACTIONS' panel with sections for 'CROWDFUNDING AT 0x0B81...', 'CREATEREQUESTS', and 'MAKEPAY'. The main editor area shows a Solidity contract named 'Crowdfunding.sol'. The contract includes a pragma statement for Solidity version 0.5.0, a 'contract Crowdfunding' definition with various state variables (contributors, manager, minContribution, target, deadline, raisedAmount, totContributors), a 'struct Request' with fields for description, payable recipient, value, completed status, totVoters, and a voters mapping, and a constructor that initializes target, deadline, and minContribution.

```
1 pragma solidity ==0.5.0 <0.9.0;
2
3
4 contract Crowdfunding {
5     mapping(address => uint256) public contributors;
6     address public manager;
7     uint256 public minContribution;
8     uint256 public target;
9     uint256 public deadline;
10    uint256 public raisedAmount;
11    uint256 public totContributors;
12
13    struct Request {
14        string description;
15        address payable recipient;
16        uint256 value;
17        bool completed;
18        uint256 totVoters;
19        mapping(address => bool) voters;
20    }
21
22    mapping(uint256 => Request) public request;
23    uint256 public numRequests;
24
25    constructor(uint256 _target, uint256 _seconds) {
26        target = _target;
27        deadline = block.timestamp + _seconds;
28        minContribution = 100 wei;
29    }
30 }
```

The bottom of the interface shows a status bar with the address '0: uint256: 1761269624' and a search bar for transactions.

WIKI PAGE LINK

- <https://github.com/htmw/F2025-Team2/wiki>



Thank You!